Real-Time Scene Understanding System

VLM-Enhanced Multi-Model Fusion Pipeline

Full Project Detail Synopsis

Camera → YOLO → MiDaS → SAM/SegFormer → VLM Narration

4-Week Build Roadmap · Research Paper Guide · VLM Integration Strategy

Stack: PyTorch · OpenCV · Ultralytics YOLO · MiDaS · SAM · SegFormer · Gradio · Claude / GPT-4V

2025 – 2026 · Student Research Project

Executive Summary

This project builds a **real-time scene understanding system** that fuses four state-of-the-art vision models — object detection (YOLOv8), monocular depth estimation (MiDaS), pixel-level segmentation (SAM / SegFormer), and vision-language narration (VLM) — into a single live pipeline running on a laptop webcam. The system continuously describes the scene in natural language, making it applicable to accessibility tools for the visually impaired, assistive robotics, surveillance, and augmented reality.

The project spans **four weeks**, each adding one model layer. A fifth, cross-cutting contribution introduced here is deep VLM integration — replacing the original text-only LLM narration with a vision-language model that directly perceives annotated camera frames, enabling richer descriptions, spatial grounding, and zero-shot scene Q&A. This upgrade also creates a three-condition ablation study that makes the work publishable.

Core Research Question	*"Can a multi-model fusion pipeline (detection + depth + segmentation + VLM) produce accurate, real-time natural language scene descriptions on commodity hardware?"*

Target Performance Metrics

Metric	Target	Notes
End-to-end FPS	≥ 10 FPS	On a mid-range laptop CPU/GPU
YOLO latency	< 30 ms / frame	YOLOv8n or YOLOv8s
MiDaS latency	< 40 ms / frame	MiDaS Small variant
SegFormer latency	< 60 ms / frame	b0-finetuned model
VLM narration latency	< 2 s / call	Called every 5–10 frames
Depth accuracy (MAE)	< 0.5 m (calibrated)	Vs. known ruler distances
Description BLEU-4	≥ 0.25 vs human captions	100-frame eval set
Human naturalness	≥ 3.8 / 5	5-person rating study

Technology Stack at a Glance

PyTorch	Deep learning runtime	**OpenCV**	Video capture + overlay
Ultralytics YOLO	Object detection	**MiDaS / Depth Anything**	Monocular depth
SAM / SegFormer	Segmentation	**Claude Vision / GPT-4V / LLaVA**	VLM narration (NEW)
Gradio	Demo UI + live feed	**ByteTrack**	Object tracking

4-Week Build Roadmap

Each week adds one model layer to the pipeline. Every stage is a self-contained deliverable that can be demonstrated independently before the next is integrated.

Week 1 — **YOLO Object Detection on Webcam**

- **Setup:** Create Python venv, install PyTorch, OpenCV, Ultralytics YOLO; verify GPU/CPU.
- **Static image first:** run YOLOv8 on a single image — understand model loading, inference, result parsing (boxes, scores, class IDs).
- **Webcam loop:** cv2.VideoCapture(0) → frame → YOLOv8 inference → draw bounding boxes + labels → cv2.imshow. Aim for 15+ FPS.
- **Study COCO classes** (80 classes) and confidence thresholds. Tune conf=0.4 as baseline.
- **Learn:** Anchor boxes, NMS (non-max suppression), IoU — understand why these exist, not just run them.

✓ **Deliverable:** Live webcam feed with labeled bounding boxes running in real time.

[PyTorch] [OpenCV] [Ultralytics YOLO]

Week 2 — **MiDaS Monocular Depth Estimation**

- **Load MiDaS** via torch.hub.load('intel-isl/MiDaS', 'MiDaS_small'). Run on a single image first.
- **Understand the output:** MiDaS outputs relative depth (not metric metres). Learn to normalise and colourize with cv2.applyColorMap.
- **Fuse with YOLO:** for each detected bounding box, sample the depth map in the box's center region → median depth → label as 'Person: ~2m'.
- **Calibration:** place a known object (e.g. 30 cm ruler) at known distances. Build a simple linear scale factor to convert relative to approximate metric depth.
- **Performance:** run YOLO and MiDaS on the same frame. Target a shared inference pipeline that doesn't bottleneck below ~10 FPS.

✓ **Deliverable:** Detected objects with approximate distance labels overlaid on live video.

[MiDaS] [PyTorch] [OpenCV]

Week 3 — **SAM / SegFormer Pixel-Level Segmentation**

- **Option A — SAM:** load sam_vit_b (lightweight). Use YOLO bounding boxes as prompts → SAM generates precise masks per object.
- **Option B — SegFormer:** load nvidia/segformer-b0-finetuned-ade-512-512 via Hugging Face. Semantic segmentation of full scene (road, sky, person, chair, wall…).
- **Recommended approach:** start with SegFormer for semantic labels (richer scene description), then add SAM for per-instance precision in Week 4's LLM prompt.
- **Build a scene parser:** extract dominant segments (by pixel area), combine with YOLO detections + MiDaS depth → structured scene_context dict.
- **Overlay:** render semi-transparent coloured masks per class on the video feed.

✓ **Deliverable:** Every pixel in the frame is labeled; structured scene dict ready for the narration layer.

[SAM] [SegFormer] [Hugging Face Transformers]

Week 4 **Intelligence Layer + LLM / VLM Narration**

- **LLM integration (baseline):** every N frames, serialize the scene dict to a prompt. Call an LLM (Claude API, GPT-4o, or local Ollama) → receive a 1–2 sentence natural language description.
- **Zone alerts:** define proximity zones (close < 1 m, medium 1–3 m, far > 3 m). Trigger a visual/audio alert when a person enters the close zone.
- **Object tracking:** add ByteTrack or DeepSORT into Ultralytics so the same person gets a consistent ID across frames — enables 'Person #1 has been in frame for 5s'.
- **Gradio UI:** wrap the whole pipeline in a Gradio interface — webcam input, live video output, narration text box, latency metrics panel.
- **Optimise:** profile inference times per module. Use threading (inference thread + display thread) to decouple I/O from compute.

✓ **Deliverable:** Live Gradio demo — webcam feed + real-time scene narration, running end-to-end on laptop.

[Gradio] [Claude / GPT-4o API] [ByteTrack]

VLM Integration — The Key Upgrade

★ NEW CONTRIBUTION — This section upgrades Week 4's text-only LLM narration to a full Vision-Language Model integration, fundamentally improving the system and creating a publishable ablation study.

Why VLMs Change Everything

The original Week 4 design feeds only a structured text dictionary to the LLM — the model is effectively *blind*, doing text completion from labels. A Vision-Language Model (LLaVA, GPT-4V, Claude Vision) actually *sees* the frame, enabling:

- Describing **what people are doing**, not just that they exist ('a person reaching for a mug' vs 'person: ~1.5 m')
- Catching objects YOLO missed, or visually correcting false positives
- Answering **natural-language questions** about the scene in real time ('is the path ahead clear?')
- Handling novel scenarios beyond COCO's 80 classes with zero additional training

Two Architectural Modes

Mode A — Hybrid (Recommended)

Keep all four pipeline stages (YOLO + MiDaS + SegFormer). Instead of serializing the scene dict to text, **overlay the YOLO bounding boxes and depth colormap directly onto the camera frame** as a visual prompt, then pass that annotated image to the VLM. The VLM now receives both rich visual context and structured metadata simultaneously.

```
# annotate_frame.py annotated = frame.copy() for box in yolo_results: cv2.rectangle(annotated, ...) depth_color = cv2.applyColorMap(depth_norm, cv2.COLORMAP_MAGMA) fused = cv2.addWeighted(annotated, 0.7, depth_color, 0.3, 0) b64 = base64.b64encode(cv2.imencode('.jpg', fused)[1]).decode() # → send b64 to VLM API every 5–10 frames
```

Mode B — Ablation (for the paper)

Run three conditions systematically to generate your comparison table: (1) scene dict text only → text LLM, (2) raw frame → VLM, (3) annotated frame (YOLO + depth overlay) → VLM. This three-way comparison is the experimental core of the research paper.

Ablation Study Design

Configuration	Model Input	Expected Strengths	Expected Weaknesses
Baseline (original)	Scene dict text	Fast, predictable, low latency	No visual grounding, hallucinations
VLM Raw (ablation A)	Raw camera frame	Rich descriptions, novel objects	Slower, may miss small objects
VLM Annotated (proposed)	Frame + YOLO/depth overlay	Best of both: grounded + rich	VLM API latency ~1–2 s per call

The proposed VLM Annotated condition is your primary hypothesis. Measuring BLEU-4, ROUGE-L, and human naturalness scores across all three conditions gives you a proper results table for Section 5 of the paper.

Research Paper Structure

Write the paper in parallel with building — document every design decision, every latency measurement, every failure. The paper writes itself if you note as you go.

Abstract

150 words max.

State the problem (real-time scene understanding without specialized sensors), your approach (4-model + VLM pipeline), and your key result (e.g. X FPS, Y% BLEU improvement for annotated-VLM over text-LLM baseline).

Introduction

1–1.5 pages.

Motivate the problem via accessibility tools for the visually impaired, assistive robotics, surveillance, or AR. Cite 3–5 related systems. End with a clear problem statement and your 3 contributions: (1) the fused pipeline, (2) annotated-frame VLM prompting strategy, (3) systematic ablation on commodity hardware.

Related Work

1 page.

Cover each component: monocular depth (MiDaS, DPT, Depth Anything), object detection (YOLO family), segmentation (SAM, SegFormer, Mask R-CNN), and vision-language models (BLIP-2, LLaVA, Claude, GPT-4V). Build this section by reading papers on arXiv.

System Architecture

1–1.5 pages.

Your pipeline diagram. Describe each module's role, input/output format, and how they connect. Describe the three VLM input modes (text dict, raw frame, annotated frame). This is essentially Week 4's scene dict architecture written formally.

Experiments

1.5–2 pages.

Dataset: record 30 minutes of indoor/outdoor footage across lighting conditions. Ground-truth: manually write what a person would say about 100 frames. Metrics: latency per module (ms), end-to-end FPS, depth MAE, BLEU-4/ROUGE-L, human naturalness rating (5 evaluators, 1–5 scale).

Results

1 page.

Tables and graphs: (1) latency breakdown table per module, (2) FPS curve vs resolution, (3) BLEU/ROUGE comparison across the three ablation conditions, (4) example output screenshots from the Gradio demo.

Discussion

0.5 page.

What works well, what fails. Common failure cases: night/low-light, fast motion blur, VLM hallucinating objects not in frame, depth accuracy drops beyond ~5 m. Honest failure analysis strengthens the paper.

Conclusion + Future Work

0.5 page.

Summarise contributions. Future work: stereo depth instead of monocular, fine-tuning the VLM on scene description data, edge deployment on Raspberry Pi, real-time audio output for the visually impaired.

Where to Submit

Venue	Type	Difficulty	Notes
arXiv preprint	Preprint server	★■■■	Free, immediate, citable — post first
CV4Accessibility @ ICCV/ECCV/CVPR	Workshop	★★■■	Perfect fit for this project's motivation
ICVGIP	Conference	★★■■	India-based, beginner-friendly
ISVC	Conference	★★■■	Broad vision scope, accepts system papers
IEEE Access	Journal	★★★■	Open-access, rigorous but not top-tier

Implementation Notes & Tips

File Structure

Organise the project as follows — one module per pipeline stage keeps things testable independently:

```
scene_understanding/
■■■ detector.py # YOLOv8 wrapper
■■■ depth_estimator.py # MiDaS wrapper + calibration
■■■ segmentor.py # SAM / SegFormer wrapper
■■■ annotator.py # Overlay boxes + depth on frame
■■■ vlm_narrator.py # VLM API call (base64 frame input)
■■■ scene_builder.py # Assemble scene dict from all modules
■■■ tracker.py # ByteTrack integration
■■■ pipeline.py # Threading: inference + display
■■■ app.py # Gradio UI entry point
```

VLM API Call — Key Code Pattern

Call the VLM every 5–10 frames on a background thread to avoid blocking the display loop. The annotated frame (YOLO boxes + depth overlay fused) is encoded as base64 JPEG and sent with a structured system prompt:

```
import anthropic, base64, cv2 client = anthropic.Anthropic() def narrate(annotated_frame, scene_dict): _, buf = cv2.imencode('.jpg', annotated_frame, [cv2.IMWRITE_JPEG_QUALITY, 75]) b64 = base64.standard_b64encode(buf).decode() system = ( 'You are a real-time scene narrator for an accessibility device. ' 'Bounding boxes and depth labels are overlaid on the image. ' 'Respond in 1-2 sentences. Be specific about distances and actions.' ) msg = client.messages.create( model='claude-opus-4-5', max_tokens=120, system=system, messages=[{'role':'user','content':[ {'type':'image','source':{'type':'base64', 'media_type':'image/jpeg','data':b64}}, {'type':'text','text':f'Scene data: {scene_dict}'} ]}] ) return msg.content[0].text
```

Threading Strategy

Decoupling inference from display is essential to hit 10+ FPS:

- **Thread 1 — Display:** reads frames from webcam, draws latest overlays, shows cv2 window at full FPS.
- **Thread 2 — Inference:** runs YOLO + MiDaS + SegFormer on every frame; updates a shared dict.
- **Thread 3 — VLM narrator:** wakes every 5–10 frames, sends annotated frame to VLM API, updates narration text.
- Use **threading.Lock()** or a queue to pass data between threads safely.
- Profile with **time.perf_counter()** around each model call; log to a CSV for the paper's latency table.

Common Failure Modes & Mitigations

Failure Mode	Symptom	Mitigation
Low-light scenes	YOLO misses objects; MiDaS noisy	Preprocess: cv2.equalizeHist or CLAHE
Fast motion blur	Bounding boxes lag; depth streaks	Lower exposure; skip frames when blur detected
VLM hallucination	Narrates objects not in frame	Ground prompt with scene dict; add 'only describe visible objects'
Depth > 5 m	Calibration breaks down	Clamp depth labels to '> 4 m — far'; document as limitation

API rate limits	Narration freezes	Cache last narration; fallback to text-LLM baseline
SegFormer slow on CPU	FPS drops below 5	Reduce input resolution to 256×256; use b0 not b2

Evaluation Framework

A rigorous evaluation plan is what separates a paper from a demo. The following plan is achievable as a solo student project over 2–3 additional weeks after the build is complete.

Dataset Collection

- Record **30 minutes** of video across: indoor (office, corridor, kitchen), outdoor (street, park), varied lighting (bright, dim, artificial).
- Sample **100 frames** uniformly: write a 1–2 sentence ground-truth caption per frame (what would a sighted person naturally say?).
- Annotate **20 frames** with known distances using a tape measure — used for depth MAE evaluation.

Automatic Metrics

Metric	What it measures	Tool
BLEU-4	N-gram overlap of narration vs. ground-truth caption	nltk.translate.bleu_score
ROUGE-L	Longest common subsequence recall	rouge-score library
Depth MAE	Mean absolute error of depth estimates vs. ruler	numpy.mean(abs(pred - gt))
End-to-end FPS	Full pipeline throughput	time.perf_counter() loop timer
Per-module latency	Bottleneck identification	time.perf_counter() per call

Human Evaluation Protocol

Ask 5 evaluators to rate 20 randomly selected narrations on two axes:

- **Naturalness (1–5):** Does it sound like something a person would naturally say?
- **Accuracy (1–5):** Does it correctly describe the scene? (Evaluator sees the frame alongside.)

Report mean ± standard deviation. Even 5 evaluators × 20 frames = 100 ratings, which is enough for a human eval table in a workshop paper.

Expected Results Layout

Condition	BLEU-4	ROUGE-L	Naturalness	FPS
Text LLM (baseline)	0.18	0.31	3.1 / 5	12.4
VLM — raw frame	0.23	0.38	3.6 / 5	9.8
VLM — annotated (proposed)	**0.29**	**0.45**	**4.1 / 5**	**9.2**

* Figures above are illustrative targets. Fill in your actual measured values.

Project Milestones & Checklist

Use this checklist to track progress across both the build and the paper.

Build

- ■ Week 1 complete: YOLOv8 live webcam, 15+ FPS, bounding boxes visible
- ■ Week 2 complete: MiDaS depth overlay, distance labels on detected objects
- ■ Week 3 complete: SegFormer masks rendered, scene dict generated per frame
- ■ Week 4 complete: Gradio demo live, text-LLM baseline narration working
- ■ VLM integration: annotated frame → Claude/GPT-4V narration on background thread
- ■ ByteTrack object tracking, consistent person IDs across frames
- ■ Threading: inference + display + VLM decoupled, stable at 10+ FPS
- ■ Calibration: depth scale factor measured against known distances

Evaluation

- ■ 30-minute video dataset recorded across environments
- ■ 100-frame ground-truth captions written
- ■ 20-frame depth ground truth measured with tape measure
- ■ Latency CSV logged per module across 500+ frames
- ■ BLEU-4 / ROUGE-L computed for all three conditions
- ■ 5-person human evaluation completed (naturalness + accuracy)

Paper

- ■ Abstract drafted (150 words, fills in X FPS, Y BLEU)
- ■ Introduction written with 3–5 related system citations from arXiv
- ■ Related work section covers depth, detection, segmentation, VLM literature
- ■ System architecture diagram drawn (pipeline + VLM modes)
- ■ Experiments section written with dataset, metrics, protocol
- ■ Results tables and FPS curve graph generated
- ■ Discussion covers failure modes honestly
- ■ arXiv preprint submitted

■ Pro tip Start writing the paper on Day 1. Keep a lab notebook (even a markdown file) where you log every design decision, every latency number, every failure and fix. When it's time to write the Experiments and Discussion sections, you'll have everything already — the paper writes itself from good notes.

Real-Time Scene Understanding

Enhanced VLM-Fusion Pipeline · v2.0

Research Paper Synopsis — With 7 Pipeline Improvements

Camera → YOLO+TRT → Depth Anything v2 → SegFormer → Pose → Temporal Buffer → Adaptive VLM

7 Targeted Improvements · New Evaluation Framework · Stronger Research Claims

Stack: PyTorch · TensorRT · Depth Anything v2 · YOLOv8-Pose · SegFormer · ByteTrack · Claude Vision · Gradio

2025 – 2026 · Student Research Project · Version 2.0

What Changed in Version 2

This synopsis supersedes v1.0. The pipeline now incorporates seven targeted improvements identified through systematic analysis of the v1.0 system's weaknesses. Each improvement is measurable, maps directly to a metric in the evaluation framework, and strengthens a specific research claim in the paper.

V1 vs V2 at a glance

Component	V1.0 (original)	V2.0 (enhanced)	Metric impact
Depth model	MiDaS Small	Depth Anything v2 Small	Depth MAE: -40%
YOLO runtime	PyTorch	TensorRT engine	Latency: -60%
Narration trigger	Every N frames	Confidence-gated (SSIM)	API calls: -70%
Narration output	Free-form text	Structured JSON + CoT	Hallucination rate measurable
Temporal stability	None	EMA smoothing window=5	Flicker events: -80%
Person description	Bounding box only	+ YOLOv8-Pose action	Description richness +2pts
Low-resource mode	No fallback	Adaptive resolution tiers	Usable on CPU-only hardware
VLM mode	Single mode	3-condition ablation	Paper: 3 result rows

Upgraded research question

V2 Research Question

"Can an adaptive, temporally-stabilised multi-model fusion pipeline — combining TensorRT-accelerated detection, metric depth estimation, semantic segmentation, action recognition, and confidence-gated VLM narration — produce accurate, flicker-free scene descriptions on commodity hardware, and does annotated-frame VLM prompting with structured chain-of-thought output measurably reduce hallucination rates compared to text-only baselines?"

New contributions summary

#	Contribution	Paper section	Novelty
C1	Temporal EMA smoothing for real-time narration stability	§3.4 / §5.3	Novel application
C2	Confidence-gated adaptive VLM invocation via SSIM	§3.5 / §5.2	Novel technique
C3	Depth Anything v2 integration with metric calibration	§3.2 / §5.1	Model upgrade + eval
C4	Structured JSON chain-of-thought VLM prompting	§3.6 / §5.4	Novel prompting strategy
C5	TensorRT YOLO + adaptive SegFormer resolution tiers	§3.3 / §5.2	Systems contribution
C6	YOLOv8-Pose action recognition fused into scene dict	§3.7 / §5.5	Novel fusion
C7	3-condition ablation: text-LLM vs VLM-raw vs VLM-annotated	§5.6	Comparative study

The 7 Pipeline Improvements

Each improvement below is self-contained — it can be built and evaluated independently. Priority order: 1→4 first (highest impact, lowest effort), then 5→7.

#1 Temporal EMA smoothing — eliminate narration flicker

Impact: High Effort: Low Perceptual quality

An exponential moving average over a sliding window of 5 frames smooths confidence scores per tracked object. Only trigger a new VLM call if the semantic content changes. Eliminates the #1 user complaint: objects appearing and disappearing every frame.

```
class TemporalBuffer: def smooth(self, detections, window=5): for det in detections:
self.scores[det.id].append(det.conf) det.conf = mean(self.scores[det.id]) # EMA return [d for d in
detections if d.conf > 0.4]
```

Research impact: Measure: flicker events per minute (VLM sentence changes / min) before vs after. Target: <3 changes/min on a static scene. This is a new metric not in v1.

#2 Confidence-gated VLM — adaptive invocation via SSIM

Impact: High Effort: Medium Systems / Cost efficiency

Compute structural similarity (SSIM) between consecutive frames and compare the object label sets. Only invoke the VLM API when the scene has changed meaningfully. On a static office scene this reduces API calls by ~70% while maintaining narration freshness.

```
def scene_changed(prev_frame, curr_frame, threshold=0.15): score = ssim(prev_frame, curr_frame,
channel_axis=2) return (1 - score) > threshold # True = invoke VLM
```

Research impact: Report: API call rate (calls/min) across scene types. 'Adaptive invocation reduced calls by 68% on static scenes while maintaining <500 ms narration lag on scene transitions.' This is a concrete systems result reviewers can verify.

#3 Depth Anything v2 — 40% better depth accuracy

Impact: High Effort: Low Depth accuracy

Drop-in replacement for MiDaS Small. Depth Anything v2 (ViT-Small) produces significantly more consistent depth maps on indoor scenes, handles reflective surfaces and edges better, and runs at comparable speed. Available on Hugging Face with a one-line swap.

```
from transformers import pipeline depth_pipe = pipeline("depth-estimation",
model="depth-anything/Depth-Anything-V2-Small-hf")
```

Research impact: Report: depth MAE (metres) on the 20-frame ruler-annotated test set. 'Depth Anything v2 reduced MAE from 0.61 m to 0.37 m — a 39% improvement.' Direct, citable, measurable. Strengthens §5.1 significantly.

#4 Structured JSON chain-of-thought — measurable hallucination reduction

Impact: High Effort: Low Narration accuracy

Force the VLM to output a structured JSON object before composing the final narration. The intermediate fields (visible_objects, scene_type, alert) can be evaluated independently against YOLO ground truth, making hallucination rate a first-class metric.

```
SYSTEM = """Respond ONLY in JSON: {"visible_objects": ["person at 1.8m"], "scene_type": "indoor",
"alert": null, "narration": "One sentence, max 20 words."}"""
```

Research impact: New metric: hallucination rate = % of VLM-listed objects not found in YOLO output. Target: <5%. Compare free-form vs structured prompting. This alone is a publishable ablation result — structured prompting has not been applied to real-time scene narration.

#5 TensorRT YOLO + adaptive SegFormer resolution tiers

Impact: Medium Effort: Medium Inference speed

Export YOLOv8n to TensorRT (.engine) for 2-3x inference speedup on NVIDIA GPUs. Add an adaptive resolution controller: SegFormer runs at 512px at >=15 FPS, 320px at 10-15 FPS, and is skipped entirely below 10 FPS. Makes the system work on CPU-only hardware.

```
model.export(format='engine') # one-time TensorRT export # Adaptive tier selection: res = {fps>=15: 512, fps>=10: 320}.get(True, None) # None = skip
```

Research impact: Latency table: YOLO PyTorch vs TensorRT (ms). FPS across hardware tiers: RTX 3050 / GTX 1650 / CPU-only. 'TensorRT reduced YOLO latency by 61%. Adaptive resolution maintained >=10 FPS on all tested hardware.' Systems contribution.

#6 YOLOv8-Pose action recognition — event-level descriptions

Impact: Medium Effort: Medium Description richness

Run YOLOv8-Pose on each detected person bounding box. Map keypoint geometry to a small action vocabulary: standing, sitting, walking, reaching, falling. Inject the action label into the scene dict and VLM prompt. Upgrades narration from object-level to event-level.

```
pose_model = YOLO('yolov8n-pose.pt') def get_action(kpts): hip_y, knee_y = kpts[11][1], kpts[13][1] return 'sitting' if abs(hip_y-knee_y)<30 else 'standing'
```

Research impact: Human eval question: 'Does the description include what the person is doing?' Rate before/after on 20 frames. Target: action accuracy >85%. 'Descriptions with action labels rated 0.9 points higher in human naturalness.' Directly supports accessibility motivation in Introduction.

#7 Adaptive resolution degradation — graceful CPU fallback

Impact: Medium Effort: Low Hardware portability

Three-tier resolution scheduling: Full (512px SegFormer, all modules) at >=15 FPS, Medium (320px SegFormer) at 10-15 FPS, Minimal (skip SegFormer, YOLO+depth only) below 10 FPS. Measures and reports description quality loss at each tier.

```
TIERS = [(15,'full',512),(10,'medium',320),(0,'minimal',None)] def get_tier(fps): return next(t for t in TIERS if fps >= t[0])
```

Research impact: New results axis: FPS x Description quality x Hardware tier. 'The system maintains >=10 FPS and BLEU-4 >= 0.21 across all tested hardware.' Makes the paper relevant to real-world deployment, not just lab demos.

Updated System Architecture

The v2 pipeline adds two new processing stages (Pose, Temporal Buffer) and upgrades three existing ones (Depth, YOLO runtime, VLM prompting). The threading model gains a fourth thread for the adaptive invocation controller.

Processing threads

Thread	Name	Runs	Modules	Output
T1	Display	Every frame	OpenCV imshow	Annotated frame to screen
T2	Inference	Every frame	YOLO-TRT, Depth Anything v2, SegFormer (adaptive), YOLOv8-Pose	Detections + depth + seg + poses
T3	Temporal buffer	Every frame	EMA smoother, scene change detector (SSIM)	Smoothed detections, change flag
T4	VLM narrator	On change (T3 flag)	Annotator, Claude Vision / GPT-4V (structured JSON)	Narration sentence + alert

Module data flow

Camera frame flows right-to-left through T2 modules in parallel. T3 reads T2 output every frame and maintains the EMA state. T4 wakes only when T3 raises the change flag, composes the annotated frame, and calls the VLM API asynchronously.

Scene dict v2 schema

The scene dict is the shared data structure between all threads:

```
{ "objects": [ {"label":"person","depth_m":1.8,"conf":0.91,"track_id":3,
"action":"walking","bbox":[x1,y1,x2,y2]} ], "scene_type": "indoor corridor", "dominant_segments":
["floor:58%","wall:22%","person:12%"], "alert_zone": "medium", "fps": 14.2, "seg_tier": "medium",
"frame_changed": true }
```

File structure v2

`scene_understanding/`	
`detector.py`	YOLOv8 TensorRT wrapper
`depth_estimator.py`	Depth Anything v2 + calibration
`segmentor.py`	SegFormer + adaptive resolution
`pose_estimator.py`	YOLOv8-Pose + action mapping [NEW]
`temporal_buffer.py`	EMA smoother + SSIM change detector [NEW]
`annotator.py`	Overlay boxes + depth colormap on frame
`vlm_narrator.py`	Structured JSON VLM API call [UPGRADED]
`scene_builder.py`	Assemble scene dict v2
`adaptive_controller.py`	Resolution tier + invocation gating [NEW]
`pipeline.py`	4-thread orchestration [UPGRADED]
`evaluate.py`	All metrics: MAE, BLEU, flicker, hallucination [NEW]
`app.py`	Gradio UI entry point

Green = new or upgraded file vs v1.0

Enhanced Evaluation Framework

V2 introduces five new metrics not present in v1. Together with the original four, this gives a comprehensive evaluation that covers accuracy, fluency, speed, stability, and hallucination — each tying directly to one of the 7 improvements.

Complete metrics table

Metric	Unit	Tool / Method	Improvement	Target
Depth MAE	metres	20-frame ruler ground truth	#3 Depth Anything v2	< 0.40 m
End-to-end FPS	frames/sec	perf_counter loop timer	#5 TensorRT	≥ 10 FPS
YOLO latency	ms/frame	perf_counter per call	#5 TensorRT	< 15 ms
SegFormer latency	ms/frame	perf_counter per call	#5 Adaptive res.	< 40 ms
VLM latency	sec/call	perf_counter API call	#2 Gated invocation	< 1.5 s
BLEU-4	0–1	nltk.translate.bleu_score	All	≥ 0.28
ROUGE-L	0–1	rouge-score library	All	≥ 0.44
Hallucination rate	% frames	VLM objects not in YOLO	#4 Structured JSON	< 5%
Flicker events	changes/min	VLM sentence change rate	#1 Temporal EMA	< 3
API call rate	calls/min	invocation counter	#2 SSIM gating	< 8 on static
Action accuracy	%	Pose vs manual label	#6 YOLOv8-Pose	≥ 85%
Human naturalness	1–5	5 raters, 20 frames	#6 Action	≥ 4.0

Ablation study design

V2 expands the original 3-condition ablation with two additional conditions that isolate the effect of temporal smoothing and structured prompting:

Condition	VLM input	Prompting	Temporal buffer	Expected BLEU-4	Expected hall
A — Baseline	Scene dict text	Free-form	None	0.18	~18%
B — VLM raw	Raw frame	Free-form	None	0.23	~12%
C — VLM annotated	Frame + overlays	Free-form	None	0.29	~8%
D — VLM annotated + JSON	Frame + overlays	Structured JSON	None	0.31	~4%
E — Full v2 (proposed)	Frame + overlays	Structured JSON	EMA window=5	0.33	~3%

* Figures are targets. Fill in measured values.

Dataset v2

- Record **45 minutes** of video: indoor (office, corridor, kitchen, stairwell), outdoor (street, park entrance), varied lighting (bright, dim, artificial, backlit).
- **150 ground-truth frames:** 100 for BLEU/ROUGE evaluation (written captions), 20 for depth MAE (ruler measurements), 30 for action accuracy (manual action labels).

- **Static scene test set:** 10 minutes of unchanging scenes to measure API call rate reduction from confidence-gated invocation.
- **Hardware test matrix:** RTX 3050 laptop GPU, GTX 1650, CPU-only (i7-1165G7). Report FPS and tier selection at each hardware level.

Research Paper Structure v2

The paper gains one new section (§3 System Design is expanded to cover all 7 improvements) and the Results section now has 6 tables instead of 3. Everything else maps directly to v1.0 structure, making the paper straightforward to write incrementally.

Abstract

150 words

State: (1) the problem — real-time scene understanding on commodity hardware without specialized sensors; (2) the approach — 5-model adaptive fusion pipeline with confidence-gated VLM narration; (3) key results — E condition achieves BLEU-4=0.33, hallucination rate 3%, 12+ FPS on RTX 3050.

Introduction

1–1.5 pages

Motivate via accessibility (visually impaired navigation), assistive robotics, and AR. Cite 3–5 related systems. Enumerate 7 contributions explicitly as bullet points — reviewers check this list against the results.

Related Work

1 page

Depth estimation (MiDaS, DPT, Depth Anything family), object detection (YOLO family), segmentation (SAM, SegFormer, Mask R-CNN), pose estimation (OpenPose, MediaPipe, YOLOv8-Pose), VLMs (BLIP-2, LLaVA, Claude, GPT-4V), temporal smoothing in video understanding.

System Design

2 pages

7 subsections — one per improvement: §3.1 Overview, §3.2 Depth Anything v2, §3.3 TensorRT, §3.4 Temporal EMA, §3.5 Confidence-gated invocation, §3.6 Structured JSON prompting, §3.7 Action recognition, §3.8 Adaptive resolution. Each subsection: motivation, method, integration point.

Experiments

1.5 pages

Dataset description, hardware matrix, ground-truth annotation protocol, all 12 metrics defined formally, ablation conditions A–E described.

Results

1.5 pages

Table 1: latency breakdown per module per hardware tier. Table 2: depth MAE MiDaS vs DA-v2. Table 3: BLEU-4/ROUGE-L conditions A–E. Table 4: hallucination rate vs prompting strategy. Table 5: flicker events vs temporal buffer window size. Table 6: action recognition accuracy.

Discussion

0.5 page

What works: structured prompting nearly eliminates hallucinations. What fails: depth accuracy degrades >4m, action recognition struggles with partial occlusion, VLM latency spikes on complex scenes. Honest failure analysis strengthens credibility.

Conclusion

0.5 page

Summarise 7 contributions. Future work: stereo depth, on-device VLM (LLaVA-1.5-7B via llama.cpp), fine-tuning on scene description corpus, Raspberry Pi edge deployment, audio output for full accessibility pipeline.

Submission targets

Venue	Type	Fit	Deadline window
arXiv preprint	Preprint	Perfect first step	Any time — submit first
CV4Accessibility @ ICCV/ECCV	Workshop	Direct application fit	~3 months before conf
ICVGIP 2025/26	Conference	Broad vision scope	Check icvgip.org
ISVC 2025/26	Conference	Systems papers welcome	Annual, ~Aug deadline
IEEE Access	Journal	Open-access, rigorous	Rolling submissions

Implementation Roadmap v2

4-week build + 2-week evaluation. Build improvements in priority order: #1 and #4 first (highest impact, lowest effort), then #3, #2, #5, #6, #7.

Week-by-week plan

Week	Focus	Improvements	Deliverable
Wk 1	YOLO + Depth upgrade	#3 Depth Anything v2 #5 TensorRT YOLO	YOLOv8-TRT + DA-v2 running at 15+ FPS, depth MAE logged
Wk 2	Stability + gating	#1 Temporal EMA #2 SSIM-gated VLM	Flicker events measurably reduced; API call rate logged on static scenes
Wk 3	Narration quality	#4 Structured JSON #6 YOLOv8-Pose	Hallucination rate measured; action labels in scene dict
Wk 4	Polish + Gradio	#7 Adaptive resolution All integration	Full pipeline in Gradio; all 12 metrics collected on test set
Wk 5	Evaluation	Ground-truth annotation Human eval (5 raters)	All tables filled in; ablation A–E complete
Wk 6	Writing	Paper draft arXiv submission	Camera-ready draft; arXiv preprint live

Master checklist

Improvements

- o #1 Temporal EMA buffer implemented and flicker metric logging
- o #2 SSIM scene-change detector + conditional VLM invocation
- o #3 Depth Anything v2 swapped in, MAE measured on 20-frame set
- o #4 Structured JSON system prompt, hallucination rate metric live
- o #5 TensorRT YOLO export, adaptive SegFormer resolution tiers
- o #6 YOLOv8-Pose integrated, action vocab mapped, action accuracy measured
- o #7 3-tier adaptive controller, CPU-only fallback tested

Evaluation

- o 45-minute video dataset recorded across 6 scene types
- o 150 ground-truth frames annotated (100 captions, 20 depth, 30 action)
- o Static scene test set (10 min) for API call rate measurement
- o Hardware matrix tested: GPU laptop, budget GPU, CPU-only
- o All 12 metrics computed and logged to CSV
- o Ablation conditions A–E run and tabulated
- o 5-person human evaluation (naturalness + accuracy, 20 frames each)

Paper

- o Abstract written with real measured numbers filled in
- o 7 contributions listed explicitly in Introduction
- o Related work covers depth, detection, segmentation, pose, VLM literature
- o System Design §3.1–§3.8 written (one subsection per improvement)
- o 6 results tables generated from eval CSV

- Discussion covers honest failure modes
- arXiv preprint submitted
- GitHub README with demo video published

Key tip Log every metric from Day 1 to a CSV file with timestamp, improvement flag, and hardware config. The 6 results tables write themselves from this one file. Every minute you spend instrumenting is worth 10 minutes of writing later.